

Multi-tier Key-Value Store

PRINCE
25M0809

Introduction :

Implemented a **RESTful** key-value store server in C++ using the [cpp-httplib](#) library .This server supports **create , read , delete** operations over http routes and stores data persistently in **PostgreSQL** database . An in-memory **LRU** based cache is also used to improve read performance . The server supports concurrency through multithreading and lock to ensure thread-safe access to shared data.

Also implemented a client to connect with server and receive commands from user to create , read and delete key-value in server .

Also implemented a multithreaded load generator that generates random requests .

System Design :

- The HTTP Server handles client requests via RESTful routes.
- The LRU cache stores recently used key-value pairs in memory.
- The PostgreSQL database persistently stores data in disk.
- Synchronization ensures safe concurrent access using locks.

System Architecture :

The system is a multi-tier key-value store designed for high-throughput reads and writes. It consists of a client layer, a C++ Server layer with in-memory cache , and a PostgreSQL persistence layer.

Incoming HTTP requests are handled by a C++ server built using httpplib, which distributes work through a thread pool, where each thread maintains its own dedicated PostgreSQL connection to avoid sharing overhead. Frequently accessed key-value pairs are stored in an LRU cache protected by a mutex lock, ensuring consistency during concurrent operations.

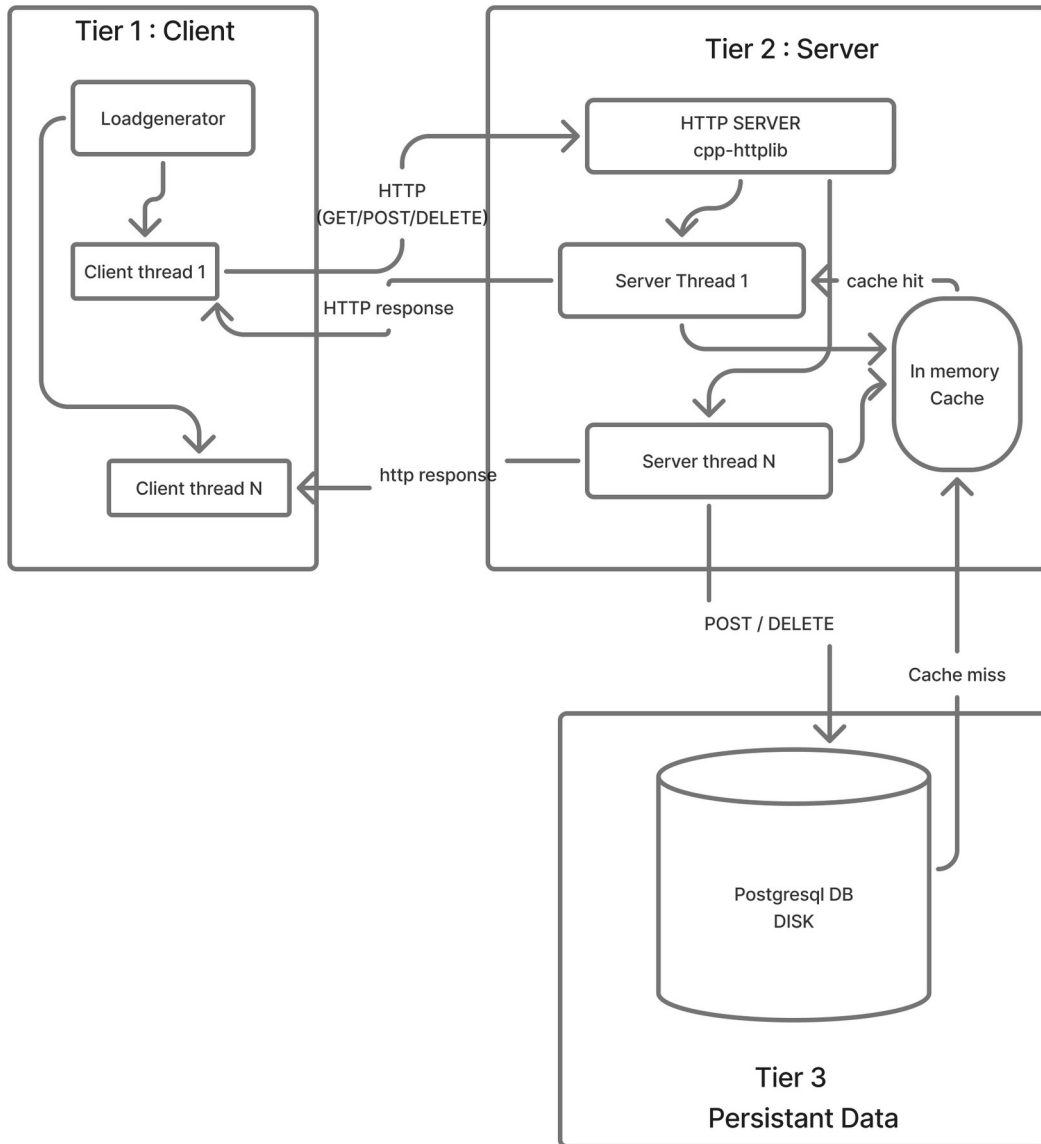
Key Features :

LRU cache : Fast reads using LRU cache, reducing database load and improving latency.

Multi-threaded : Multi-threaded processing with thread-local DB connections for scalability.

Consistency : Consistency between cache and DB maintained on insert, fetch update, and delete operations.

PostgreSQL database : Reliable persistent storage with ACID guarantee .



Implementation Details :

Server →

- Implemented using cpp-httplib library .
- 3 REST endpoints :
 - POST (/) - Create a new key-value pair .
 - GET (/:key) – Retrieve a value by key from server .
 - DELETE (/:key) – Delete a key-value pair .
- Each request is handled by a thread from threadpool implemented in cpp-httplib.

Database →

- PostgreSQL using libpq library .
- Connection established using Pqconnectdb() .
- Used PqexecParams() to prevent SQL injection .

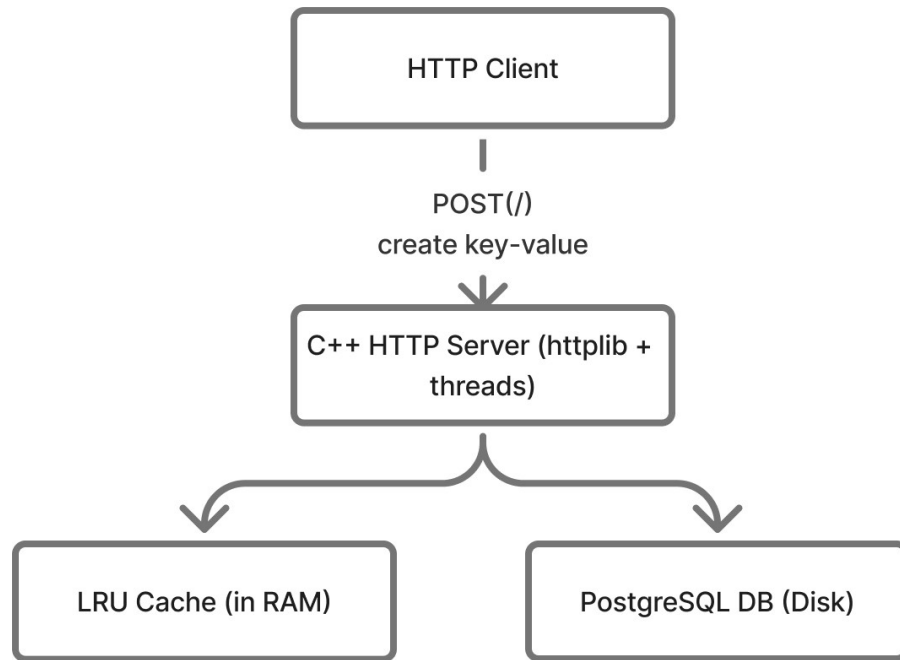
Cache →

- Implemented using doubly linked list and unordered map .
- Provides nearly $O(1)$ access to key-values stored in cache .

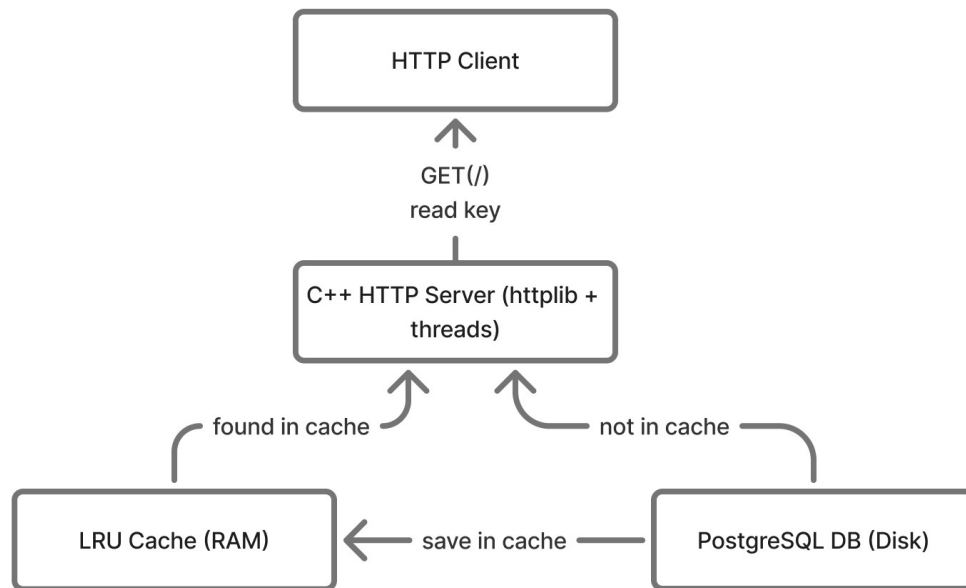
Client and Loadgenerator →

- User interactive client function to connect with server .
- Multi-threaded loadgenerator to generate random requests parallelly .

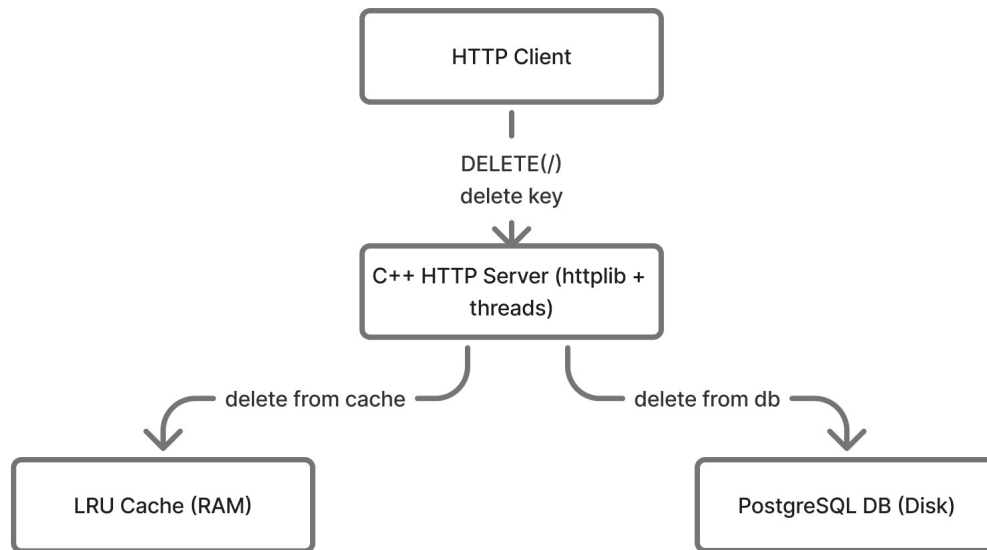
Create →



Read →



Delete →



2 Load Generator Design :

- Implemented in **C++ using pthreads** to simulate concurrent clients.
- Uses httplib Client to send HTTP requests to the server.
- Each thread repeatedly sends GET or POST requests until a timeout signal is triggered.

2.1 Workload Type :

a) GET Popular :

- Continuously fetches a small set of popular keys, resulting in frequent cache hits.
-
- Stresses the in-memory LRU cache and tests CPU utilization under heavy read traffic.
-
- Leads to high throughput and sub-millisecond latency, with CPU becoming the primary bottleneck.

b) PUT-ALL :

- Performs repeated write and update operations on a large key space, eliminating cache benefits.
-
- Stresses PostgreSQL write performance and disk I/O, since every write must persist to storage.
-
- Shows low throughput and high latency due to disk-bound behavior, with disk utilization exceeding 93%.

Workload	Path	Main Bottleneck
GET-POPULAR	Cache hit path	CPU (cache hit rate)
PUT-ALL	Cache miss + DB write path	Disk I/O (PostgreSQL write performance)

Load Test Setup :

- **Load Generator** : Closed-loop with varying number of concurrent clients
- **Load Levels**: 1, 2, 3, 5, 8, 10 , 12 concurrent clients
- **Duration**: Each test ran for 5+ minutes to reach steady state
- **CPU Isolation**: Load generator and server run on separate CPU cores
- **Metrics Collection**: Throughput (req/sec), latency (ms), CPU utilization (%), disk utilization (%)

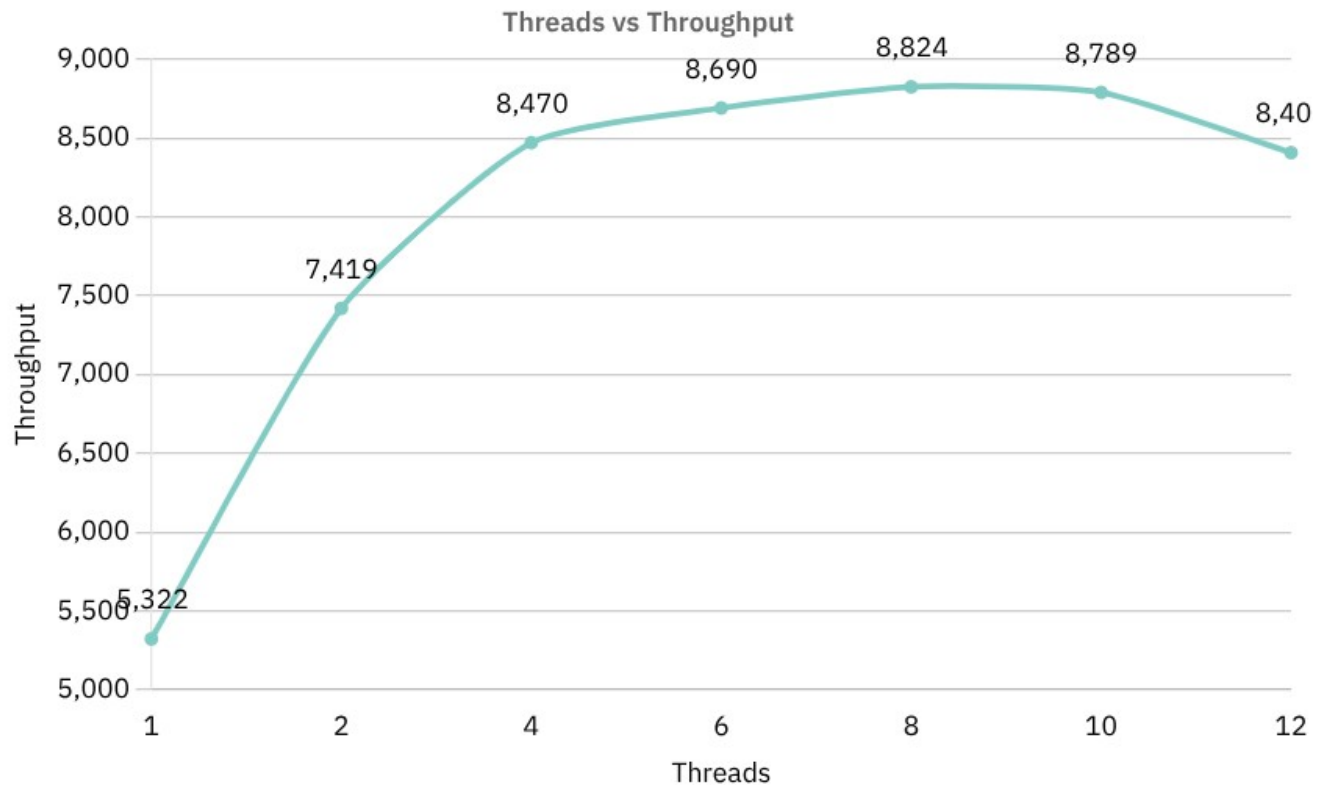
Load Test Results :

GET-POPULAR Workload :

The GET-POPULAR workload tests the system's ability to handle repeated read requests for a small set of popular keys. This workload primarily benefits from cache hits and tests in-memory performance.

Clients	Response(ms)	CPU Utilization %	Through put (req /s)	disk utilization %
1	1.01	18	977	47
2	1.03	38	1926	90
4	1.28	62	3112	91
5	1.4	75	3554	92
7	1.65	94	4205	93
9	1.94	95	4621	92
11	2.31	97	4741	92
13	3	96	4303	93

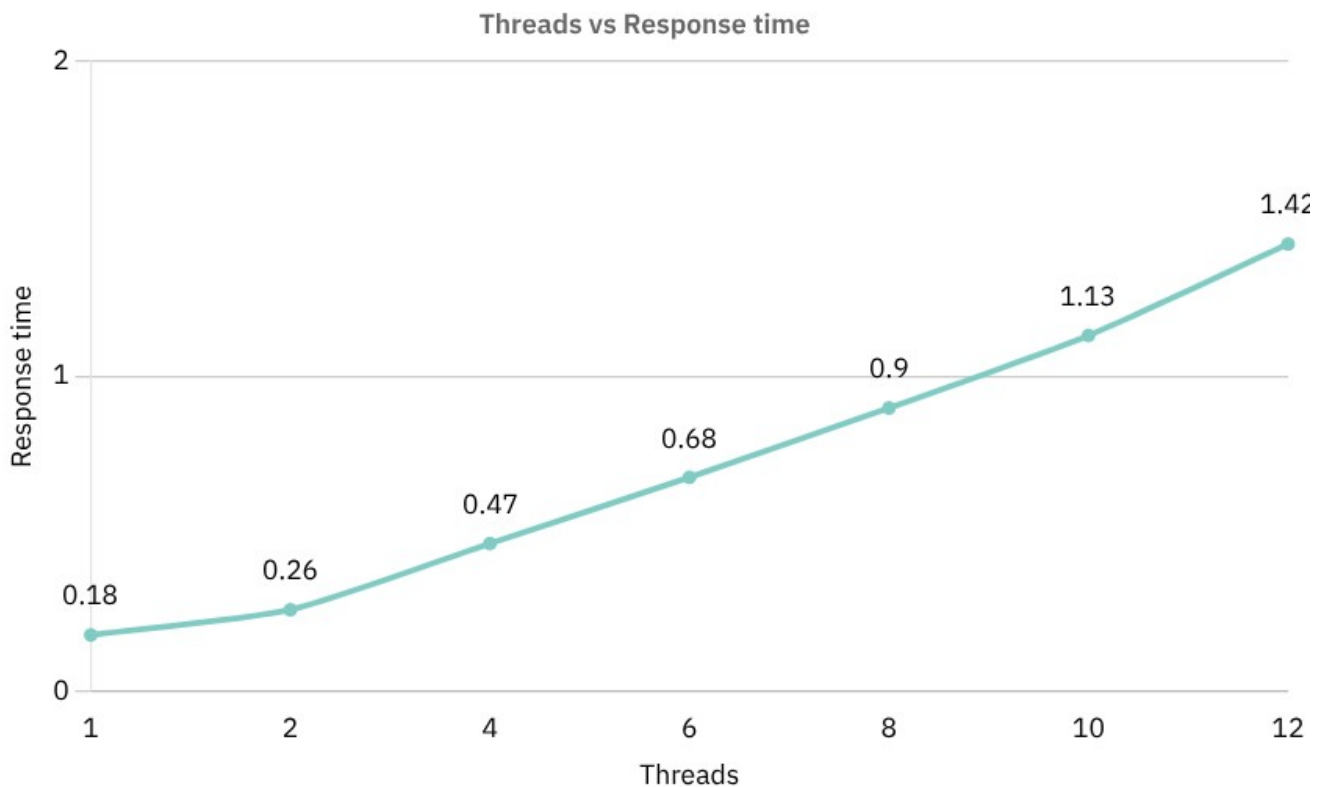
Throughput Analysis :



Observations:

- Throughput increases sharply from 5322 req/sec (1 client) to 7419 req/sec (2 clients)
- Plateaus around 8600-8820 req/sec for 8-10 clients
- Maximum throughput of 8824 req/sec achieved at 8 clients
- System reaches saturation beyond 8 clients with minimal throughput gains

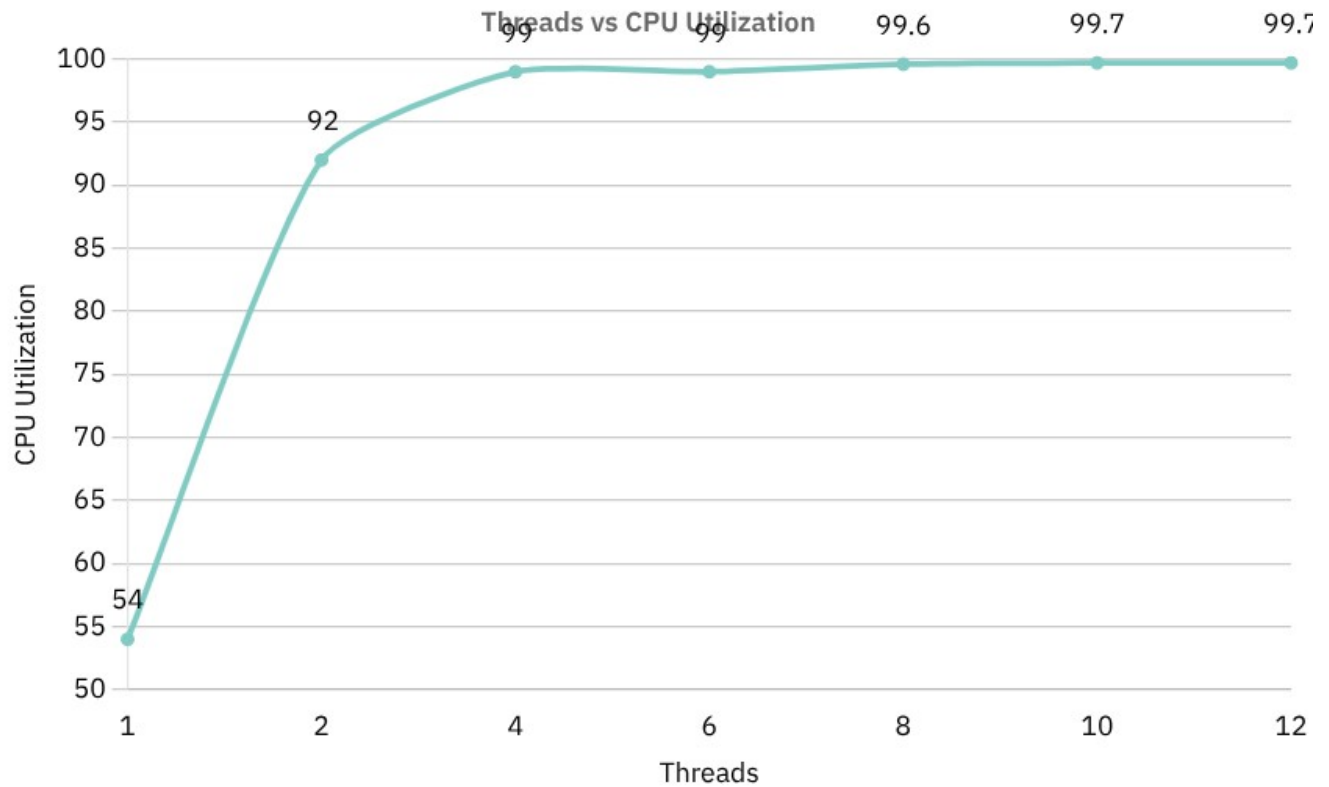
Response Time Analysis :



Observations:

- Response time starts at 0.18 ms for single client .
- Increases gradually to 0.68 ms at 6 clients
- Shows steeper increase beyond 8 clients: 0.9 ms
- Reaches 1.42 ms at 12 clients
- Latency degradation indicates queueing effects and resource contention at high load

CPU Utilization Analysis :



Observations:

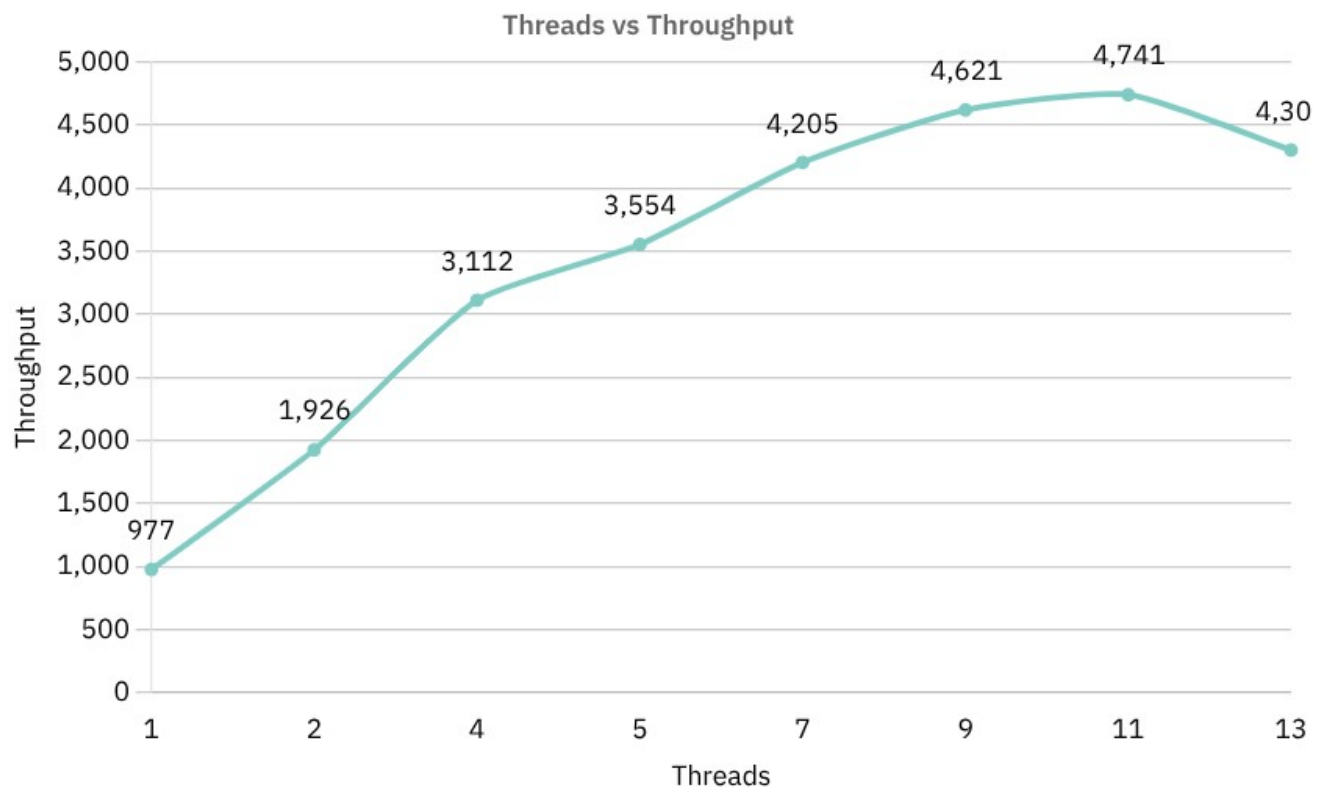
- CPU utilization jumps from 54% (1 client) to 92% (2 clients)
- Reaches 99 % at 3 clients and stabilizes around 99%
- Maximum CPU utilization: 99% at 12 clients
- CPU is the primary bottleneck for GET-POPULAR workload
- Plateau indicates system has reached CPU capacity

PUT-ALL Workload :

The PUT-ALL workload tests the system's write performance by continuously creating key-value pairs. This workload stresses the database write path and disk I/O subsystem.

Clients	Response time (ms)	CPU utilization	Throughput	Disk Utilization
1	0.18	54	5322	1
2	0.26	92	7419	1
4	0.47	99	8470	1
6	0.68	99	8690	1
7	0.9	99.6	8824	1
10	1.13	99.7	8789	1
12	1.42	99.7	8407	1

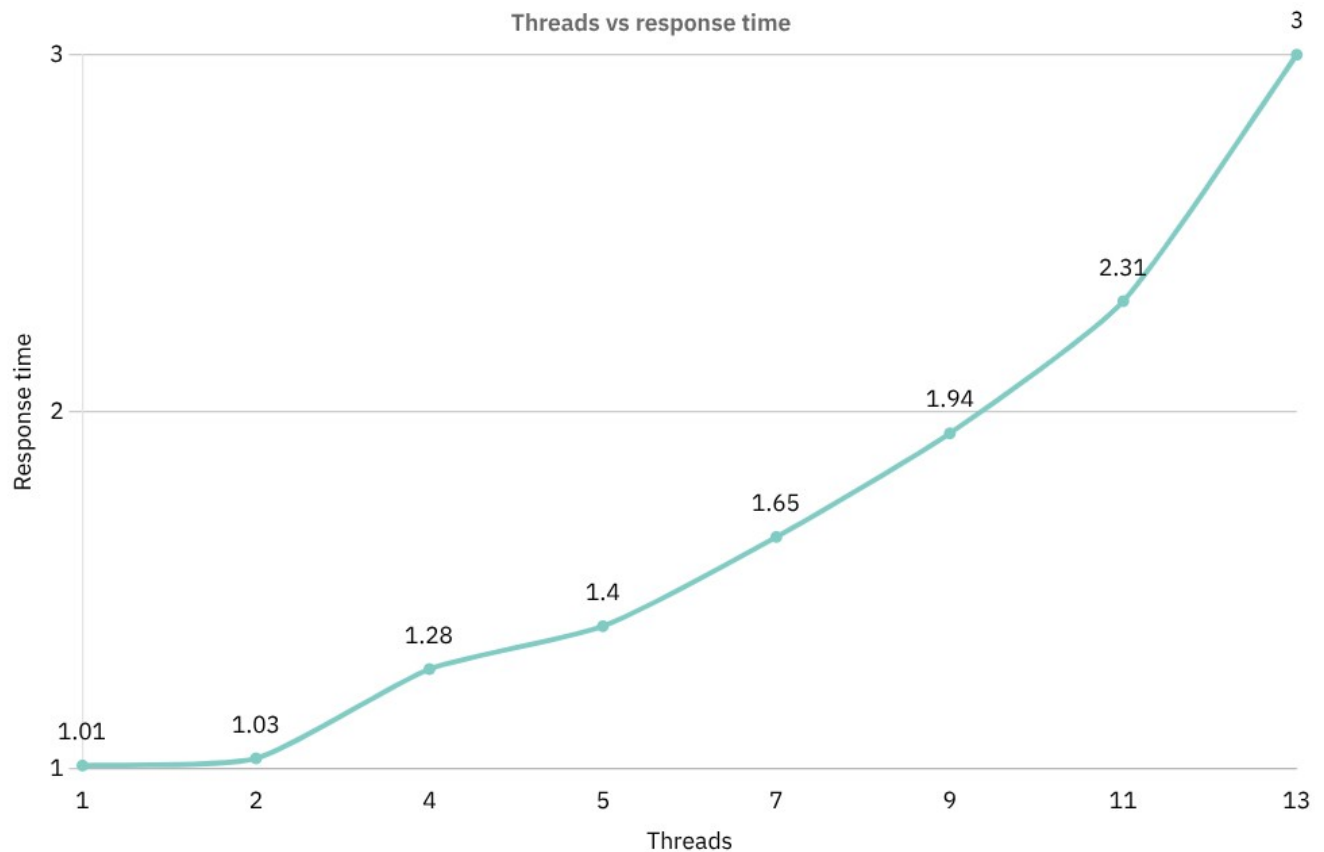
Throughput Analysis :



Observations:

- Throughput increases from 977 req/sec (1 client) to 3112 req/sec (4 clients)
- Continues growing to 4741 req/sec at 11 clients
- Plateaus around 4600-4700 req/sec for 9-11 clients
- Maximum throughput of 4741 req/sec at 11 clients
- Lower throughput compared to GET-POPULAR due to disk write overhead

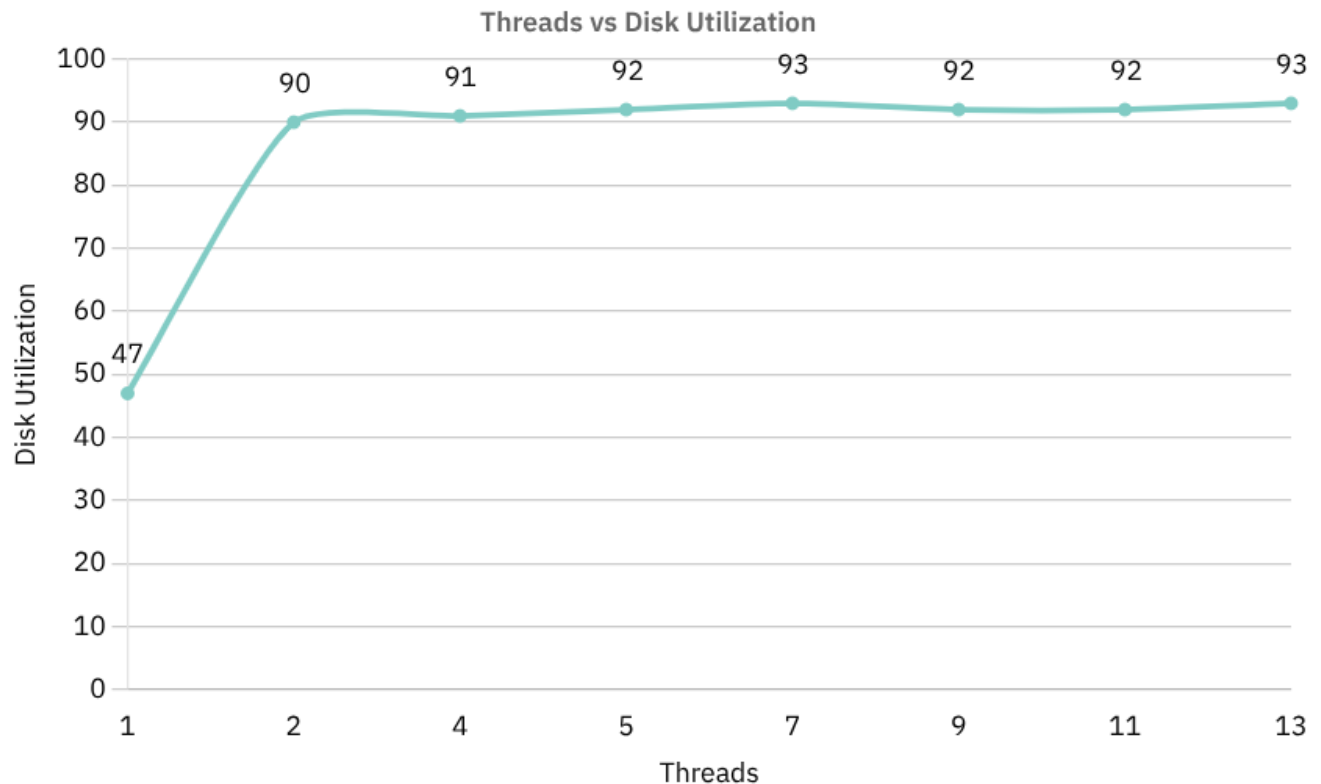
Response time Analysis :



Observations:

- Response time starts at 1.01 ms for single client .
- Increases to 3 ms at 13 clients
- Much higher latencies compared to GET-POPULAR workload

Disk Utilization Analysis :



Observations:

- Disk utilization starts at 47% for single client
- Quickly reaches 90% at 2 clients .
- Remains consistently high (92+) across all load levels from 5 clients onwards .
- Maximum disk utilization: 93% at 13 clients
- Disk I/O is the primary bottleneck for PUT-ALL workload

GITHUB Link →

[Key – Value store](#)

Files →

Server - server.cpp

cmd – g++ server.cpp -o server -lpq
./server #LRU cache capacity

Client – client.cpp

cmd – g++ client.cpp -o client
./client

Load generator (put all work load)– putall.cpp

cmd – g++ putall.cpp -o putall
./putall

Load generator (get popular work load)– getpopular.cpp

cmd – g++ getpopular.cpp -o getpopular
./ getpopular

Client commands :

Create kv-pair = create key value-size value

ex - create 10 6 prince

Read kv-pair = read key

ex – read 10

Delete kv-pair = delete key

ex – delete 10

References →

- cpp-httplib - <https://github.com/yhirose/cpp-httplib>
- PostgreSQL libpq - <https://www.postgresql.org/docs/>